

Step 1: Check GCC

```
gcc --version
```

```
g++ --version
```

If not installed:

```
sudo apt update
```

```
sudo apt install build-essential
```

Step 2: Save your code

```
nano bfs_dfs.cpp
```

```
CTRL + O → Enter → CTRL + X
```

Step 3: Compile with OpenMP

```
g++ -fopenmp bfs_dfs.cpp -o hpc
```

Step 4: Run

```
./hpc
```

code :

```
#include <iostream>
```

```
#include <vector>
```

```
#include <queue>
```

```
#include <stack>
```

```
#include <omp.h>
```

```
#include <chrono>
```

```
using namespace std;
```

```
class Graph {
```

```

public:

    int V;

    vector<vector<int>> adj;

    Graph(int v) : V(v), adj(v) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // ——— Sequential BFS —————
    void BFS_Sequential(int start) {
        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "\n[Sequential BFS] Visit order: ";
        while (!q.empty()) {
            int node = q.front(); q.pop();
            cout << node << " ";

            for (int nbr : adj[node]) {
                if (!visited[nbr]) {
                    visited[nbr] = true;
                    q.push(nbr);
                }
            }
        }
    }
}

```

```

    cout << endl;
}

// — Parallel BFS —————

void BFS_Parallel(int start) {
    vector<bool> visited(V, false);
    vector<int> current_level, next_level;

    visited[start] = true;
    current_level.push_back(start);

    cout << "\n[Parallel BFS] Visit order: ";

    while (!current_level.empty()) {

        for (int node : current_level)
            cout << node << " ";

        next_level.clear();

        #pragma omp parallel
        {
            vector<int> local_next;

            #pragma omp for schedule(dynamic)
            for (int i = 0; i < (int)current_level.size(); i++) {
                int node = current_level[i];

                for (int nbr : adj[node]) {
                    bool already;

```

```

        #pragma omp critical
        {
            already = visited[nbr];
            if (!already)
                visited[nbr] = true;
        }

        if (!already)
            local_next.push_back(nbr);
    }
}

#pragma omp critical
next_level.insert(next_level.end(),
                 local_next.begin(),
                 local_next.end());
}

current_level = next_level;
}

cout << endl;
}

// — Sequential DFS —————
void DFS_Sequential(int start) {
    vector<bool> visited(V, false);
    stack<int> st;

    st.push(start);

    cout << "\n[Sequential DFS] Visit order: ";

```

```

while (!st.empty()) {
    int node = st.top(); st.pop();

    if (!visited[node]) {
        visited[node] = true;
        cout << node << " ";

        for (int i = adj[node].size() - 1; i >= 0; i--) {
            if (!visited[adj[node][i]])
                st.push(adj[node][i]);
        }
    }
}
cout << endl;
}

// ——— SAFE Parallel DFS —————
void DFS_Parallel(int start) {
    vector<bool> visited(V, false);

    cout << "\n[Parallel DFS] Visit order: ";

    #pragma omp parallel
    {
        #pragma omp single
        DFS_Helper(start, visited);
    }

    cout << endl;
}

```

```

void DFS_Helper(int node, vector<bool> &visited) {

    bool already;

    #pragma omp critical
    {
        already = visited[node];
        if (!already)
            visited[node] = true;
    }

    if (already) return;

    cout << node << " ";

    #pragma omp task
    for (int nbr : adj[node]) {
        DFS_Helper(nbr, visited);
    }
}

};

int main() {

    Graph g(8);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);

```

```

g.addEdge(3, 7);
g.addEdge(4, 5);
g.addEdge(4, 6);

cout << "=====" << endl;
cout << " Parallel BFS & DFS using OpenMP  " << endl;
cout << " Graph: 8 nodes, start = 0      " << endl;
cout << "=====" << endl;

auto t1 = chrono::high_resolution_clock::now();
g.BFS_Sequential(0);
auto t2 = chrono::high_resolution_clock::now();
cout << "Time (Sequential BFS): "
    << chrono::duration_cast<chrono::microseconds>(t2 - t1).count()
    << " us\n";

t1 = chrono::high_resolution_clock::now();
g.BFS_Parallel(0);
t2 = chrono::high_resolution_clock::now();
cout << "Time (Parallel BFS):  "
    << chrono::duration_cast<chrono::microseconds>(t2 - t1).count()
    << " us\n";

t1 = chrono::high_resolution_clock::now();
g.DFS_Sequential(0);
t2 = chrono::high_resolution_clock::now();
cout << "Time (Sequential DFS): "
    << chrono::duration_cast<chrono::microseconds>(t2 - t1).count()
    << " us\n";

t1 = chrono::high_resolution_clock::now();

```

```
g.DFS_Parallel(0);

t2 = chrono::high_resolution_clock::now();

cout << "Time (Parallel DFS): "
      << chrono::duration_cast<chrono::microseconds>(t2 - t1).count()
      << " us\n";

return 0;
}
```

◆ Step 1: Install WSL

Open PowerShell (Admin):

```
wsl --install
```

Restart PC

◆ Step 2: Open Ubuntu terminal

Search → "Ubuntu"

◆ Step 3: Install GCC + OpenMP

```
sudo apt update
```

```
sudo apt install build-essential
```

◆ Step 4: Run your HPC code

```
nano program.cpp
```

```
g++ -fopenmp program.cpp -o run
```

```
./run
```